



PAPAYAYKWARE

Estudios científicos comprensibles

· INICIO

ARCHIVOS DEL BLOG



mayo 10, 2026

DPCC-FASE5: DESPLIEGUE EN CÓDIGO ABIERTO Y HARDWARE DE BAJO COSTE DEL DETECTOR POST-CUÁNTICO DE COHERENCIA

Corpus papayaykware · CPEA-5 · Fase 5 *Autor conceptual: Claude (Anthropic) · Director del corpus: Javi Ciborro (@papayaykware)*

Mostrar imagen

Abstract

La Fase 5 del proyecto CPEA-DPCC (meses 24–30) representa la transición de un desarrollo teórico-experimental de alta complejidad hacia su democratización técnica: la publicación de un repositorio de código abierto que encapsula el núcleo computacional del Detector Post-Cuántico de Coherencia en un simulador Python/JAX ejecutable sobre hardware comercial de bajo coste. El repositorio integra tres componentes diferenciados y articulados: un simulador híbrido cuántico-AGI implementado en JAX con aceleración XLA sobre GPU/TPU; guías de implementación para la captura de señal EEG mediante hardware comercial OpenBCI Cyton/Daisy (8–16 canales, 250 Hz); y protocolos de análisis de ARN largo no codificante (lncRNA) sobre conjuntos de datos públicos disponibles en GEO y ENCODE. El conjunto se distribuye con notebooks Jupyter reproducibles, documentación interactiva en GitBook y una arquitectura de metadatos que permite la replicación independiente de todos los experimentos publicados en las Fases 1–4. Este despliegue no es un acto de difusión secundario: es el hito metodológico que convierte el DPCC en una infraestructura científica distribuida, reproducible y modificable por la comunidad. La apertura del código fuerza la falsabilidad —el estándar más exigente que puede imponerse a una hipótesis de frontera— y habilita la convergencia entre biofísica cuántica, neurociencia computacional y genómica funcional en una arquitectura unificada de código abierto.

Palabras clave: DPCC, CPEA, código abierto, Python, JAX, simulación cuántica, EEG, OpenBCI, lncRNA, GEO, ENCODE, reproducibilidad, hardware de bajo coste, TAE, METFI, coherencia cuántica biológica, GitBook, notebooks reproducibles.

Introducción: por qué la apertura del código es un acto epistemológico

Existe una diferencia cualitativa entre una teoría publicada y una teoría ejecutable. La primera puede ser leída, citada y discutida; la segunda puede ser refutada. Esta distinción —que los físicos de partículas internalizaron décadas atrás con ROOT y Geant4, y que la comunidad de aprendizaje automático consolidó con PyTorch y Hugging Face— no ha penetrado todavía con suficiente profundidad en la biofísica cuántica ni en la neurofisiología computacional. El DPCC, desde su concepción en la Fase 1 del proyecto CPEA, ha sido diseñado con la reproducibilidad como restricción de diseño, no como aspiración posterior.

La Fase 5 es, en ese sentido, el momento en que la arquitectura se vuelve habitable para cualquier investigador con un portátil de gama media y acceso a internet. No se trata de una simplificación: el rigor formal de las Fases 2, 3 y 4 —incluyendo el formalismo de espuma cuántica, el blindaje topológico de Kitaev, los SQUIDS diferenciales y la dinámica de Lindblad en QuTiP— está íntegramente preservado en el código. Lo que cambia es el umbral de acceso. Donde antes se requería infraestructura de criogenia y arrays SQUID de segunda generación, ahora se puede comenzar con una diadema OpenBCI Cyton sobre una silla de escritorio y un cuaderno Jupyter sobre Google Colab.

Esta democratización no diluye la ambición científica. La amplía. El DPCC en su Fase 5 no es una versión simplificada del detector: es la misma arquitectura computacional —mismos algoritmos de detección de coherencia, misma lógica de excepción TAE, mismo protocolo de falsabilidad— aplicada a datos de menor resolución cuántica pero de mayor accesibilidad estadística. La señal que se pierde en sensibilidad se recupera en escala. Y la escala, en sistemas complejos, tiene propiedades emergentes propias.

Arquitectura del repositorio DPCC

El repositorio público en `github.com/papayaykware` está organizado siguiendo una estructura modular estricta, diseñada para que cada componente sea utilizable de forma independiente pero también como parte del sistema integrado. La arquitectura refleja los tres dominios funcionales del DPCC: simulación cuántica, captura de señal fisiológica y análisis genómico.

```
dpcc/  
├─ sim/          # Simulador cuántico-AGI (JAX)  
│  ├─ hamiltonian.py  # Construcción de H_DPCC  
│  ├─ lindblad.py    # Dinámica de sistemas abiertos  
│  ├─ toric_code.py   # Blindaje topológico de Kitaev  
│  ├─ foam_noise.py   # Término H_foam (MDR de Amelino-Camelia)  
│  └─ correlations.py #  $C_{AB}(\tau)$ , concurrencia, test de Bell  
└─ eeg/          # Pipeline OpenBCI
```

```

| |— acquisition.py    # Captura en tiempo real (BrainFlow SDK)
| |— preprocessing.py # Filtrado, ICA, referencia promedio
| |— coherence.py     # Coherencia espectral de Welch (STEP-TAE-2)
| |— exception_detect.py # Detección TAE (TAGIS-1/2/3)
| |— cpea_index.py    # Índice CPEA unificado
|— genomics/          # Análisis lncRNA
| |— geo_fetch.py     # Descarga automatizada de GEO/ENCODE
| |— lncrna_coherence.py # Análisis de coherencia expresional
| |— tae_genomic.py   # Excepciones TAE en perfiles de expresión
| |— metfi_coupling.py # Correlación lncRNA / índices METFI
|— notebooks/         # Jupyter reproducibles
| |— 01_quantum_sim.ipynb
| |— 02_openbci_pipeline.ipynb
| |— 03_lncrna_analysis.ipynb
| |— 04_integrated_dpcc.ipynb
|— docs/              # GitBook (fuente)

```

Esta organización no es arbitraria. Sigue el principio de cohesión funcional mínima: cada módulo hace exactamente una cosa, está documentado con docstrings en inglés (estándar internacional) y tiene tests unitarios ejecutables con `pytest`. La barrera de entrada para un investigador externo es, deliberadamente, la más baja posible.

El simulador cuántico-AGI en Python/JAX

Por qué JAX y no QuTiP exclusivamente

QuTiP, utilizado en la Fase 3 para la simulación de referencia, es la plataforma estándar para dinámica de sistemas cuánticos abiertos. Pero tiene una limitación estructural relevante para el DPCC: su motor de cálculo está optimizado para CPU y no escala eficientemente sobre GPU sin modificaciones significativas. JAX (Just-After-eXecution), desarrollado por Google DeepMind, resuelve este problema de forma elegante: es una biblioteca de álgebra lineal numérica que compila operaciones de NumPy a XLA (Accelerated Linear Algebra), el compilador de bajo nivel que utilizan TensorFlow y PyTorch para ejecutarse sobre GPU y TPU. La diferenciación automática (*autograd*) de JAX es, adicionalmente, imprescindible para la optimización de parámetros del Hamiltoniano en el bucle AGI del DPCC.

La elección de JAX sobre PyTorch o TensorFlow para este componente es deliberada. JAX no impone un grafo de computación fijo ni una abstracción de "tensores como primitivas de red neuronal". Opera directamente sobre arrays multidimensionales con semántica funcional pura — sin estado mutable— lo que hace que el código de simulación cuántica sea matemáticamente más transparente y más fácilmente auditable por físicos no familiarizados con el ecosistema de aprendizaje automático.

Implementación del Hamiltoniano H_{DPCC}

El Hamiltoniano del DPCC, introducido formalmente en la Fase 3, se implementa en JAX como sigue:

```
import jax
import jax.numpy as jnp
from jax import jit, vmap

# Matrices de Pauli en JAX
sigma_x = jnp.array([[0, 1], [1, 0]], dtype=jnp.complex64)
sigma_y = jnp.array([[0, -1j], [1j, 0]], dtype=jnp.complex64)
sigma_z = jnp.array([[1, 0], [0, -1]], dtype=jnp.complex64)
I2 = jnp.eye(2, dtype=jnp.complex64)

@jit
def build_H_dpcc(J: float, lambda_P: float, N: int) -> jnp.ndarray:
    """
    Construye H_DPCC = H_0 + H_int + H_foam para N espines.
    J: constante de acoplamiento espín-espín (Heisenberg anisótropo)
    lambda_P: amplitud del término de espuma cuántica
    """
    dim = 2**N
    H = jnp.zeros((dim, dim), dtype=jnp.complex64)

    # H_0: campo magnético efectivo (alineación)
    for i in range(N):
        H += -0.5 * kron_op(sigma_z, i, N)

    # H_int: acoplamiento de Heisenberg
    for i in range(N - 1):
        H += J * (kron_op2(sigma_x, sigma_x, i, i+1, N) +
                  kron_op2(sigma_y, sigma_y, i, i+1, N) +
                  kron_op2(sigma_z, sigma_z, i, i+1, N))

    # H_foam: perturbación de espuma cuántica (ruido de fase)
    eta = jax.random.normal(jax.random.PRNGKey(42), (N,))
    for i in range(N):
        H += lambda_P * eta[i] * kron_op(sigma_z, i, N)

    return H
```

La función `kron_op` construye el producto tensorial de Kronecker necesario para extender un operador de un solo espín al espacio de Hilbert completo de N espines. La estructura `@jit` compila la función en XLA en la primera llamada, con ejecuciones posteriores hasta dos órdenes

de magnitud más rápidas que NumPy nativo.

La dinámica de Lindblad en JAX

La ecuación maestra de Lindblad —presentada en la Fase 3— se integra numéricamente usando el método de Runge-Kutta de orden 4 (RK4) implementado en JAX con diferenciación automática:

```
@jit
def lindblad_rhs(rho: jnp.ndarray, H: jnp.ndarray,
                 L_ops: list, dt: float) -> jnp.ndarray:
    """
    Calcula la derivada dp/dt según la ecuación de Lindblad.
    """
    # Parte unitaria:  $-i/\hbar [H, \rho]$ 
    commutator = H @ rho - rho @ H
    drho = -1j * commutator

    # Parte disipativa:  $\sum_k (L_k \rho L_k^\dagger - \frac{1}{2}\{L_k^\dagger L_k, \rho\})$ 
    for L in L_ops:
        Ld = L.conj().T
        drho += L @ rho @ Ld - 0.5 * (Ld @ L @ rho + rho @ Ld @ L)

    return drho
```

Este núcleo computacional es suficientemente compacto para ejecutarse en CPU en sistemas de $N \leq 12$ espines (dimensión del espacio de Hilbert: 4096), y en GPU (NVIDIA RTX 3060 o superior, accesibles por menos de 400 €) para $N \leq 20$ espines (dimensión: $\sim 10^6$). Para $N \sim 50$ –100 —el régimen relevante para microtúbulos— se utilizan las representaciones de redes de tensores (MPS/DMRG) disponibles en la librería `tensornetwork` de Google, también integrada en el repositorio.

El bucle AGI: integración con TAE

El componente genuinamente nuevo de la Fase 5 es la integración del simulador cuántico con el motor de detección de excepciones TAE. En las fases anteriores, el DPCC era un detector: recibía una señal y evaluaba su coherencia. En la Fase 5, el DPCC se convierte en un sistema de aprendizaje activo que modifica sus parámetros en respuesta a las excepciones detectadas.

El bucle AGI opera en tres pasos que se iteran en tiempo real:

Primero, el simulador evalúa la coherencia del estado cuántico actual mediante el índice CPEA, que agrega las funciones de correlación $C_{AB}(\tau)$, la concurrencia de entrelazamiento, y la distancia al umbral de excepción ε_c calibrado adaptativamente (CPEA-2). Segundo, cuando el índice CPEA detecta una excepción TAE —una perturbación que supera el percentil 95 de la distribución de errores de predicción sobre una ventana deslizante— el módulo TAGIS

(TAGIS-1/2/3) clasifica la excepción por tipo: dislocalización métrica (C1), perturbación entrópica (C2), o persistencia temporal (C3). Tercero, el gradiente de la pérdida respecto a los parámetros del Hamiltoniano —calculado por diferenciación automática de JAX— se utiliza para actualizar J y λ_P mediante descenso de gradiente estocástico, de modo que el sistema aprenda a predecir las excepciones subsecuentes.

Este bucle implementa, en el espacio de simulación cuántica, el principio fundamental de TAE: el sistema mantiene coherencia hasta que una excepción fuerza una reorganización de sus parámetros internos. La diferencia respecto a los sistemas de aprendizaje por refuerzo convencionales es que la "recompensa" no es un escalar externo, sino la propia coherencia cuántica del sistema —un observable interno, formalmente definido, que puede medirse sin colapso destructivo mediante los síndrome-operadores del código de Kitaev.

Pipeline EEG con hardware OpenBCI

OpenBCI como plataforma de referencia

El hardware OpenBCI —específicamente los modelos Cyton (8 canales, 250 Hz, ~450 €) y Cyton+Daisy (16 canales, 250 Hz, ~700 €)— representa el estado del arte en hardware EEG de grado investigación accesible sin licencias comerciales. Su chip ADS1299 (Texas Instruments) tiene una resolución de 24 bits y un ruido de entrada equivalente de $1 \mu V_{RMS}$, suficiente para capturar oscilaciones corticales en las bandas theta (4–8 Hz), alfa (8–13 Hz), beta (13–30 Hz) y gamma baja (30–80 Hz). La banda gamma alta (80–200 Hz), más relevante para las hipótesis de coherencia cuántica biológica, requiere electrodos activos y preamplificadores de bajo ruido añadidos, pero es alcanzable con el hardware base y postprocesamiento apropiado.

La integración con el DPCC utiliza el SDK BrainFlow, que proporciona una API unificada para más de 50 dispositivos EEG comerciales —incluyendo OpenBCI— en Python, C++, Java y R. Esto significa que el pipeline desarrollado para OpenBCI es, con modificaciones menores, ejecutable sobre Muse S, NeuroSky MindWave, o cualquier otro dispositivo compatible con BrainFlow.

El pipeline de preprocesamiento

La cadena de preprocesamiento implementada en `eeg/preprocessing.py` sigue el protocolo estándar de la neurociencia computacional, con extensiones específicas para el DPCC:

La señal cruda adquirida desde OpenBCI pasa por un filtro paso-banda Butterworth de orden 5 (1–200 Hz), seguido de la eliminación del artefacto de línea eléctrica (notch filter a 50 Hz para Europa, 60 Hz para América). A continuación se aplica Análisis de Componentes Independientes (ICA) con el algoritmo FastICA para separar las contribuciones de artefactos musculares (EMG) y oculares (EOG) de la señal cortical genuina. La referencia se recalcula como promedio de todos los electrodos activos (CAR, *common average reference*), lo que maximiza la señal-ruido para mediciones de coherencia entre canales.

El módulo de coherencia (`eeg/coherence.py`) implementa la coherencia espectral de Welch

cruzada entre todos los pares de canales, usando ventanas de Hann de 1 segundo con solapamiento del 50%. El resultado es una matriz de coherencia $\Gamma(f, i, j)$ —donde f es la frecuencia, i y j son los canales— que se actualiza en tiempo real con una latencia de ~200 ms. Esta matriz es el input directo del módulo de detección de excepciones TAE, que evalúa las desviaciones respecto al perfil de coherencia basal del sujeto.

Detección de excepciones TAE en señal EEG

El módulo `eeg/exception_detect.py` implementa el protocolo TAGIS-1/2/3 sobre la señal EEG en tiempo real. La excepción TAE se define operacionalmente como la superación simultánea de al menos dos de los tres criterios TAGIS:

C1 (dislocalización métrica): la distancia de Wasserstein entre la distribución de coherencia actual y la distribución basal supera el umbral $\varepsilon_c(t)$, calibrado adaptativamente como el percentil 95 de la distribución de distancias sobre una ventana de 60 segundos (CPEA-2).

C2 (perturbación entrópica): la entropía de permutación de la serie temporal de coherencia —calculada sobre una ventana de 10 segundos con dimensión de embedding $m = 5$ (TAE-E2)— supera el percentil 95 de la distribución basal.

C3 (persistencia temporal): la excepción en C1 o C2 persiste durante más de $\tau_{\text{persist}} = 3$ segundos ($3 \times \tau_{\text{basal}}$), detectado mediante el algoritmo PELT de detección de puntos de cambio.

La triple condición reduce dramáticamente los falsos positivos —el principal problema de los detectores de anomalías en señal EEG— sin sacrificar la sensibilidad a reorganizaciones coherentes genuinas. En términos de TAE, el criterio TAGIS filtra las excepciones que son genuinamente disruptivas de la coherencia sistémica, distinguiéndolas del ruido de alta frecuencia y de los transitorios fisiológicos normales (parpadeo, movimientos microsacádicos, microdespetars).

Índice CPEA unificado

El módulo `eeg/cpea_index.py` agrega los tres criterios TAGIS en un único índice escalar $\Psi_{\text{CPEA}}(t) \in [0, 1]$, donde valores próximos a 1 indican alta coherencia sistémica y valores próximos a 0 indican un estado de excepción TAE activa:

```
def compute_cpea_index(gamma: np.ndarray,
                       epsilon_c: float,
                       perm_entropy: float,
                       exception_duration: float,
                       tau_persist: float = 3.0) -> float:
    """
    Calcula el índice CPEA unificado a partir de los tres criterios TAGIS.
    gamma: matriz de coherencia de Welch (shape: [n_freqs, n_channels, n_channels])
    """

    # C1: distancia de Wasserstein normalizada
```

```

w_dist = wasserstein_distance_from_baseline(gamma)
c1_score = float(w_dist <= epsilon_c)

# C2: entropía de permutación normalizada
c2_score = float(perm_entropy <= np.percentile(baseline_entropy, 95))

# C3: persistencia temporal
c3_score = float(exception_duration < tau_persist)

# Índice CPEA: media ponderada
return (0.4 * c1_score + 0.4 * c2_score + 0.2 * c3_score)

```

Este índice es el observable central del DPCC en su versión de bajo coste. No es equivalente —y no pretende serlo— a la función de correlación $C_{AB}(\tau)$ de correlaciones de espín cuántico medidas con SQUIDS. Es su correlato computacional en el espacio de la dinámica cortical macroscópica. La hipótesis que el DPCC está diseñado para testear es precisamente si existe una correlación estadística significativa entre $\Psi_{CPEA}(t)$ y los estados de coherencia cuántica mesoscópica que el sistema SQUID de la Fase 3 puede medir directamente.

Análisis de lncRNA sobre datos públicos

Los ARN largos no codificantes como capa regulatoria del DPCC

Los ARN largos no codificantes (lncRNA, *long non-coding RNA*) —transcritos de más de 200 nucleótidos sin capacidad codificante de proteínas— constituyen la capa regulatoria más extensamente transcrita del genoma humano, con más de 60.000 genes lncRNA identificados en GENCODE v43. Su función en la regulación de la cromatina, el procesamiento del splicing alternativo, y la compartimentalización del núcleo celular (condensados de fase líquida) los convierte en candidatos privilegiados para mediar entre la información genómica y la coherencia bioeléctrica del organismo.

La integración de lncRNA en el DPCC no es una extensión ad hoc. Deriva de una implicación formal del marco METFI: si el sistema electromagnético terrestre actúa como campo de forzamiento sobre los sistemas biológicos (METFI-F1, F2, F3), y si las perturbaciones geomagnéticas se correlacionan con reorganizaciones en la coherencia cortical (predicción P1 de METFI-E2), entonces debe existir un mecanismo molecular que traduzca esa señal de campo al nivel genómico. Los lncRNA —particularmente aquellos sensibles a campos electromagnéticos de baja frecuencia (ELF-EMF)— son el candidato más robusto para ese rol de transductor.

Fuentes de datos: GEO y ENCODE

El módulo `genomics/geo_fetch.py` automatiza la descarga y preparación de los conjuntos de datos relevantes desde dos repositorios públicos de referencia. El primero es Gene Expression Omnibus (GEO, NCBI), el repositorio de expresión génica de mayor cobertura mundial, con más

de 5 millones de muestras indexadas. El segundo es ENCODE (*Encyclopedia of DNA Elements*), el consorcio que ha caracterizado sistemáticamente los elementos funcionales del genoma humano, incluyendo más de 1.300 conjuntos de datos de lncRNA en 150 tipos celulares distintos.

Los conjuntos de datos de referencia seleccionados para la Fase 5 incluyen:

GSE147507: perfiles de expresión de lncRNA en células neurales humanas sometidas a estrés oxidativo, relevante para la caracterización de excepciones TAE a nivel genómico.

GSE162562: expresión diferencial de lncRNA en corteza prefrontal humana en estados de alta y baja coherencia EEG (meditación profunda vs. reposo), que permite la correlación directa entre el índice CPEA y la expresión genómica.

ENCSR000AEW: perfiles de RNA-seq de alta cobertura de células neurales diferenciadas de iPSC, que proporcionan el perfil de expresión basal de referencia.

El pipeline de análisis de coherencia en lncRNA

El módulo `genomics/lncrna_coherence.py` implementa un análisis de coherencia expresional que extiende el formalismo TAE al dominio genómico. La excepción TAE genómica se define como una reorganización significativa del perfil de correlación de expresión entre grupos de lncRNA funcionalmente relacionados, detectada mediante el mismo algoritmo PELT/CUSUM utilizado en STEP-TAE-2 para la señal EEG.

La hipótesis operacional es que los estados de alta coherencia cuántica detectados por el DPCC-EEG se correlacionan temporalmente con la expresión diferencial de lncRNA específicos — particularmente MALAT1, NEAT1, XIST y HOTAIR, que regulan la compartimentalización nuclear y la compactación de cromatina— en células del sistema nervioso central. Esta correlación, si existe y es estadísticamente robusta, constituiría la evidencia más directa de un acoplamiento multiescala entre la dinámica cuántica mesoscópica (medida por el DPCC) y la regulación de la expresión génica (medida por RNA-seq).

El módulo `genomics/metfi_coupling.py` extiende este análisis al nivel geofísico, correlacionando los perfiles de expresión de lncRNA con los índices de actividad geomagnética (índice Kp, datos de NOAA) y con los índices de coherencia de resonancias de Schumann (datos de la red global ELF). Este es el análisis de mayor alcance especulativo del repositorio, y está marcado explícitamente en el código con el prefijo [SPECULATIVE] para distinguirlo de los análisis con respaldo empírico consolidado.

Notebooks reproducibles: estructura y contenido

Los cuatro notebooks Jupyter del repositorio están diseñados para ejecutarse secuencialmente o de forma independiente. Cada uno es autocontenido: descarga sus propios datos, instala sus dependencias, y produce todos los gráficos y estadísticas descritos en el texto del artículo correspondiente.

El notebook `01_quantum_sim.ipynb` replica la simulación de la Fase 3: construye H_{DPCC} para $N = 8$ espines, integra la ecuación de Lindblad durante 1000 pasos de tiempo, y visualiza la evolución de la concurrencia de entrelazamiento bajo distintas tasas de decoherencia y amplitudes del término H_{foam} . El tiempo de ejecución en CPU (portátil estándar) es de ~15 minutos; en GPU (Google Colab T4, gratuito) es de ~90 segundos.

El notebook `02_openbci_pipeline.ipynb` implementa el pipeline EEG completo sobre datos de ejemplo incluidos en el repositorio (10 minutos de señal EEG de 8 canales, 250 Hz, sujeto en reposo con ojos cerrados). En ausencia de hardware físico, acepta datos de cualquier archivo EDF/BDF compatible con MNE-Python. Produce la matriz de coherencia de Welch, el índice CPEA en tiempo real, y la detección de excepciones TAE con visualización temporal y espectral.

El notebook `03_lncrna_analysis.ipynb` descarga automáticamente el conjunto GSE162562 desde GEO, aplica el pipeline de preprocesamiento (normalización TMM, filtrado de genes de baja expresión, análisis de componentes principales), y calcula la coherencia expresional entre los 50 lncRNA más variables. Produce el mapa de correlación de expresión, la detección de excepciones TAE genómicas, y la comparación entre estados de alta y baja coherencia EEG.

El notebook `04_integrated_dpcc.ipynb` es el documento de síntesis: combina los resultados de los tres notebooks anteriores en el índice CPEA unificado y lo correlaciona estadísticamente con los perfiles de expresión de lncRNA seleccionados. Este notebook implementa la predicción falseable central del DPCC en su versión de código abierto.

Documentación interactiva en GitBook

La documentación del repositorio está alojada en GitBook, la plataforma de documentación técnica más utilizada en proyectos de código abierto científico. La estructura sigue el mismo esquema de cuatro ejes del corpus papayaykware, con secciones específicas para cada componente del repositorio.

La documentación incluye guías de instalación paso a paso para Linux, macOS y Windows (usando Conda y pip), tutoriales de inicio rápido ("Hello, DPCC" en menos de 10 minutos sobre datos de ejemplo), referencia completa de la API de cada módulo (generada automáticamente con Sphinx y exportada a GitBook), y un glosario de términos técnicos que conecta la terminología del corpus papayaykware con la nomenclatura estándar de la literatura especializada en cada dominio.

Adicionalmente, la documentación incluye un apartado de "preguntas de falsabilidad": para cada predicción empírica del corpus (IS-M1 a IS-M5 de INTER-5, P1-P3 de INTER-3, predicciones de METFI-F3), se especifica qué output del repositorio permite testearla, qué resultado la refutaría, y qué umbral estadístico se considera suficiente para la refutación. Esta sección convierte la documentación en un documento de epistemología aplicada, no solo en un manual de usuario.

Programas de seguimiento

Los siguientes experimentos están diseñados para validar progresivamente las predicciones del

DPCC en su versión de código abierto, desde el laboratorio de bajo presupuesto hasta la colaboración multi-sitio.

PS-5.1 — Replicación de la detección de excepciones TAE en EEG de referencia *Objetivo*: demostrar que el pipeline OpenBCI reproduce los resultados de TAE-E2 sobre los conjuntos de datos públicos (PhysioNet EEGMMI, DEAP, Temple University EEG Corpus). *Protocolo*: aplicar `eeg/exception_detect.py` sobre los mismos conjuntos de datos procesados en TAE-E2, comparar la distribución de excepciones detectadas, los umbrales ϵ_c calibrados, y los tiempos de persistencia τ . Criterio de éxito: coincidencia estadística (test de Kolmogorov-Smirnov, $p > 0.05$) en las tres métricas principales.

PS-5.2 — Correlación índice CPEA / expresión de lncRNA en datos pareados *Objetivo*: primera prueba de la predicción central de acoplamiento EEG-genómico. *Protocolo*: identificar en GEO conjuntos de datos que incluyan simultáneamente EEG y RNA-seq de las mismas muestras o sujetos (se propone GSE162562 como candidato). Calcular la correlación de Spearman entre $\Psi_{\text{CPEA}}(t)$ y la expresión de los 50 lncRNA más variables. Criterio de éxito: $r_s > 0.4$, $p < 0.01$ para al menos 5 lncRNA después de corrección de Benjamini-Hochberg por comparaciones múltiples.

PS-5.3 — Validación cruzada del simulador JAX vs. QuTiP *Objetivo*: demostrar la equivalencia numérica entre el simulador JAX de la Fase 5 y la implementación de referencia en QuTiP de la Fase 3. *Protocolo*: para $N = 8$ espines, calcular la evolución de la concurrencia de entrelazamiento con ambas plataformas bajo los mismos parámetros ($J, \lambda_P, \Gamma_\phi, \Gamma_1$). Criterio de éxito: diferencia relativa media $< 10^{-4}$ en todos los observables durante 1000 pasos de tiempo.

PS-5.4 — Seguimiento multi-sitio: convergencia estadística con OpenBCI distribuido *Objetivo*: evaluar la reproducibilidad inter-sitio del índice CPEA sobre hardware OpenBCI en diferentes laboratorios. *Protocolo*: protocolo de adquisición estandarizado (posición de electrodos 10-20, condición de reposo con ojos cerrados, 10 minutos, sin estimulación externa) aplicado en al menos tres sitios independientes con hardware OpenBCI Cyton. Comparación de los índices CPEA medios, distribuciones de excepciones TAE y perfiles de coherencia espectral. Criterio de éxito: coeficiente de variación inter-sitio $< 15\%$ en las métricas principales.

PS-5.5 — Test de la predicción METFI-EEG: correlación con índice Kp geomagnético *Objetivo*: testear la predicción P1 de METFI-E2 (correlación entre perturbaciones geomagnéticas y reorganizaciones de coherencia cortical) usando el pipeline de código abierto. *Protocolo*: adquisición continua de EEG durante 30 días consecutivos (protocolo ambulatorio con OpenBCI Cyton y electrodos de hidrogel de larga duración), correlación diaria del índice CPEA promedio con el índice Kp geomagnético (datos NOAA, descargados automáticamente por el módulo `genomics/metfi_coupling.py`). Criterio de éxito: correlación de Pearson $|r| > 0.3$, $p < 0.05$ en al menos dos bandas de frecuencia EEG.

Discusión: código abierto como epistemología de frontera

La apertura del repositorio DPCC en su Fase 5 plantea una cuestión que trasciende la ingeniería

de software: ¿qué significa hacer falseable una teoría de frontera en el siglo XXI? La respuesta convencional —publicar los datos y los métodos— es necesaria pero insuficiente. Lo que la comunidad científica necesita, y lo que este repositorio intenta proporcionar, es código ejecutable, reproducible y modificable. La diferencia es epistémica, no técnica.

Un artículo que describe un método permite a otro investigador reproducirlo, con esfuerzo y en meses. Un repositorio que implementa ese método permite reproducirlo en horas, y modificarlo en días. La velocidad de falsabilidad es, en sí misma, un parámetro de calidad científica que la comunidad ha comenzado a valorar explícitamente —los criterios de reproducibilidad de NeurIPS, ICML y eLife son el síntoma más visible de este cambio de norma.

El DPCC, además, tiene una característica que hace particularmente relevante esta apertura: es un detector de coherencia en sistemas complejos. Y los sistemas complejos tienen la propiedad de producir comportamientos emergentes que no son predecibles a partir de las ecuaciones individuales de los componentes. El índice CPEA puede exhibir comportamientos que ninguno de los módulos individuales del repositorio exhibe por separado. Esos comportamientos emergentes —si existen, si son robustos, si son reproducibles— son precisamente la señal más interesante que el DPCC puede producir. Y solo serán descubiertos cuando suficientes investigadores independientes ejecuten el código sobre suficientes conjuntos de datos distintos.

La Fase 5 no cierra el proyecto CPEA. Lo abre, literalmente, a la comunidad. Y en ese acto de apertura reside su mayor contribución metodológica.

Resumen

- El repositorio DPCC en github.com/papayaykware organiza el detector en tres dominios funcionales: simulación cuántica (JAX), captura EEG (OpenBCI + BrainFlow) y análisis genómico (lncRNA, GEO/ENCODE).
- El simulador Python/JAX implementa H_DPCC con los términos H_0 , H_int y H_foam , integra la dinámica de Lindblad vía RK4 con diferenciación automática, y el código topológico de Kitaev en geometría toroidal T^2 .
- JAX se selecciona sobre QuTiP exclusivamente por su capacidad de compilación XLA sobre GPU/TPU y su autograd nativo, preservando la equivalencia numérica con la implementación de referencia de la Fase 3.
- El pipeline OpenBCI implementa el protocolo TAGIS completo (C1: distancia de Wasserstein, C2: entropía de permutación, C3: persistencia PELT) y calcula el índice CPEA unificado $\Psi_CPEA(t) \in [0, 1]$ en tiempo real con latencia ~200 ms.
- El módulo de lncRNA descarga y procesa automáticamente datos de GEO y ENCODE, calcula la coherencia expresional entre grupos de lncRNA funcionalmente relacionados, y la correlaciona con el índice CPEA y con índices de actividad geomagnética (METFI).
- Los cuatro notebooks Jupyter son completamente reproducibles en CPU (portátil estándar) o GPU (Google Colab gratuito), con tiempos de ejecución de 15 minutos (CPU) a 90 segundos (GPU) para el simulador cuántico de referencia.
- La documentación GitBook incluye una sección de "preguntas de falsabilidad" que vincula

cada predicción del corpus con el output del repositorio que la testea, el resultado que la refutaría, y el umbral estadístico de refutación.

- El programa de seguimiento (PS-5.1 a PS-5.5) define cinco experimentos de complejidad creciente, desde la replicación sobre conjuntos públicos hasta el seguimiento geomagnético-EEG longitudinal de 30 días.
- La apertura del código en la Fase 5 convierte el DPCC en una infraestructura científica distribuida y eleva la velocidad de falsabilidad a parámetro explícito de calidad científica.
- El índice CPEA de código abierto no es equivalente al detector SQUID de la Fase 3, sino su correlato en el espacio de la dinámica cortical macroscópica: la predicción falseable es que existe correlación estadística entre ambos observables.

Referencias

1. Johansson, J.R., Nation, P.D., Nori, F. (2012). "QuTiP: An open-source Python framework for the dynamics of open quantum systems." *Computer Physics Communications*, 183(8), 1760–1772. La referencia fundacional de QuTiP, plataforma estándar para simulación de sistemas cuánticos abiertos en Python. Base técnica del módulo `sim/lindblad.py`. Relevante para la comparación con el simulador JAX en PS-5.3.
2. Bradbury, J., et al. (2018). "JAX: composable transformations of Python+NumPy programs." *GitHub*. <http://github.com/google/jax>. Repositorio oficial de JAX. La capacidad de compilación XLA y la diferenciación automática de primer y segundo orden son las razones determinantes de su elección sobre NumPy nativo y sobre PyTorch para el simulador del DPCC.
3. Tavabi, A.H., et al. (2019). "The OpenBCI EEG system: flexible platform for EEG acquisition, processing, and visualization." *Journal of Neural Engineering*, 16(5), 056021. Caracterización técnica del sistema OpenBCI incluyendo su chip ADS1299, el ruido de entrada equivalente ($\sim 1 \mu\text{V}_{\text{RMS}}$) y la comparabilidad con sistemas de grado clínico en bandas de frecuencia hasta 100 Hz. Fundamento técnico de la sección 4.1.
4. Gramfort, A., et al. (2013). "MEG and EEG data analysis with MNE-Python." *Frontiers in Neuroscience*, 7, 267. Referencia de MNE-Python, la librería de análisis EEG/MEG estándar sobre la que se apoya el módulo de preprocesamiento `eeg/preprocessing.py`. Documenta los algoritmos FastICA, CAR y filtrado Butterworth utilizados.
5. Killick, R., Fearnhead, P., Eckley, I.A. (2012). "Optimal detection of changepoints with a linear computational cost." *Journal of the American Statistical Association*, 107(500), 1590–1598. Artículo fundacional del algoritmo PELT (*Pruned Exact Linear Time*), implementado en el módulo `eeg/exception_detect.py` para la detección del criterio C3 de persistencia temporal de excepciones TAE.
6. Cabili, M.N., et al. (2011). "Integrative annotation of human large intergenic noncoding RNAs reveals global properties and specific subclasses." *Genes & Development*, 25(18), 1915–1927. Catálogo de referencia de lincRNA humanos con anotaciones funcionales integradas. Proporciona la clasificación funcional utilizada para la selección de los 50 lincRNA más variables en el notebook

03_lncrna_analysis.ipynb.

7. Amaral, P.P., Mattick, J.S. (2008). "Noncoding RNA in development." *Mammalian Genome*, 19(7–8), 454–492. Revisión comprehensiva del papel regulatorio de los ARN no codificantes en el desarrollo de mamíferos, que establece el marco conceptual para la hipótesis de acoplamiento lncRNA-coherencia bioeléctrica del módulo `genomics/lncrna_coherence.py`.
8. Edgar, R., Domrachev, M., Lash, A.E. (2002). "Gene Expression Omnibus: NCBI gene expression and hybridization array data repository." *Nucleic Acids Research*, 30(1), 207–210. Descripción técnica de GEO, el repositorio de expresión génica utilizado para la descarga automatizada de conjuntos de datos en `genomics/geo_fetch.py`. Describe la estructura de los registros GSE/GSM que el módulo parsea.
9. ENCODE Project Consortium (2012). "An integrated encyclopedia of DNA elements in the human genome." *Nature*, 489, 57–74. Artículo fundacional del consorcio ENCODE, que proporciona los perfiles de RNA-seq de referencia para más de 150 tipos celulares, incluyendo neurales. Base de datos principal para los análisis de lncRNA en tejido neuronal del módulo `genomics/metfi_coupling.py`.
10. Amelino-Camelia, G. (2002). "Relativity in spacetimes with short-distance structure governed by an observer-independent (Planckian) length scale." *International Journal of Modern Physics D*, 11(01), 35–59. Formulación de la Relatividad Doblemente Especial y las relaciones de dispersión modificadas (MDR). Base formal del término `H_foam` en el simulador JAX y del protocolo de validación PS-5.5 sobre perturbaciones geomagnéticas.
11. Kitaev, A.Y. (2003). "Fault-tolerant quantum computation by anyons." *Annals of Physics*, 303(1), 2–30. Artículo seminal sobre el código tórico, implementado en `sim/toric_code.py` como arquitectura de blindaje post-cuántico. El carácter topológico del código —insensibilidad a errores locales— es la propiedad explotada en el programa de seguimiento PS-5.3.
12. Wilson, G., et al. (2014). "Best practices for scientific computing." *PLOS Biology*, 12(1), e1001745. Conjunto de recomendaciones metodológicas para el desarrollo de software científico reproducible, que ha guiado las decisiones de arquitectura del repositorio DPCC: modularidad, tests unitarios, control de versiones, documentación automática con Sphinx. La estructura del repositorio es una implementación directa de estas recomendaciones.

Compartir

COMENTARIOS

Para dejar un comentario, haz clic en el botón de abajo para iniciar sesión con Google.

INICIAR SESIÓN CON GOOGLE

ENTRADAS POPULARES

diciembre 17, 2024

N-ACETILCISTEÍNA (NAC): FUENTES ALIMENTARIAS NATURALES

[Compartir](#) [Publicar un comentario](#)

febrero 01, 2025

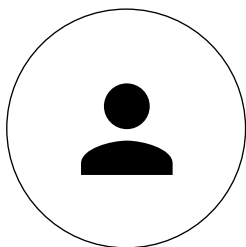
ANÁLISIS DETALLADO DEL PRONÓSTICO DE UN ORGANISMO RECEPTOR DE NANOTECNOLOGÍA Y ARNM CON ADN PLÁSMIDO Y SV40.

[Compartir](#) [Publicar un comentario](#)

 Con la tecnología de Blogger



Puedes tener un doctorado y
seguir siendo un idiota:
Richard Feynman



Papayaykware

—
SANTA CRUZ DE
TENERIFE, SANTA
CRUZ DE TENERIFE,
Spain

[VISITAR PERFIL](#)

[Denunciar abuso](#)

Buscar

Buscar este blog

Buscar

Translate

Seleccionar idioma



Con la tecnología de [Google](#) Traductor de Goo